WEB APPLICATION VULNERABILITY ASSESSMENT

Week 7 | Burp Suite + OWASP ZAP | DVWA Local Lab

Prepared by	Akash Sanuria
Intern ID	DG/SEPTEMBER/CYBER/171
Assessment date	20 October 2026
Test environment	DVWA | http://127.0.0.1:4280
Document type	Practical training report | 14 pages

Scope: an intentionally vulnerable application running locally. No public website or production environment was tested.

01 | Executive Summary

This Week 7 exercise assessed a locally hosted Damn Vulnerable Web Application (DVWA) using Burp Suite Community Edition and OWASP ZAP. Manual tests demonstrated SQL injection, reflected cross-site scripting (XSS), and stored XSS in DVWA configured at Low security. Additional exercises covered request interception, authentication failure behavior, parameter changes, and endpoint discovery.

Assessment highlights

Area	Observed outcome
SQL injection	A crafted input returned multiple DVWA user records in the training application.
Reflected XSS	Injected JavaScript produced an alert when the reflected response was rendered.
Stored XSS	A guestbook message containing script was stored and executed on page load.
Authentication	Incorrect password was rejected with a visible “Login failed” message.
Parameter tampering	Changing id=1 to id=2 returned Gordon Brown; not evidence by itself of broken authorization.
ZAP Spider	Completed at 100%; 122 URLs found and 57 nodes added.

Important interpretation

This is a training-lab report. Severity ratings are qualitative and lab-specific; they are not production risk scores. Automated ZAP alerts require manual validation before being described as confirmed vulnerabilities.

02 | Scope, Authorization & Constraints

In scope

• DVWA instance at http://127.0.0.1:4280 on the local machine.

• Authentication and demonstration pages: /login.php, /vulnerabilities/sqli/, /vulnerabilities/xss_r/, and /vulnerabilities/xss_s/.

• Passive analysis and ZAP Spider discovery on the local target.

Out of scope

• Any external domains, public hosts, third-party services or production applications.

• Persistence, destructive payloads, credential harvesting or exfiltration.

Environment

Component	Use
Kali Linux on VirtualBox	Testing workstation
DVWA Docker container	Intentionally vulnerable target
Burp Suite Community Edition	Proxy, HTTP history and Repeater
OWASP ZAP 2.17.0	Manual Explore, Spider and passive alerts
Browser	Access to DVWA and observation of XSS alerts

Testing was restricted to the user-controlled training environment. The ZAP active scan was interrupted earlier; no claim of a completed active scan is made.

03 | Methodology & Workflow

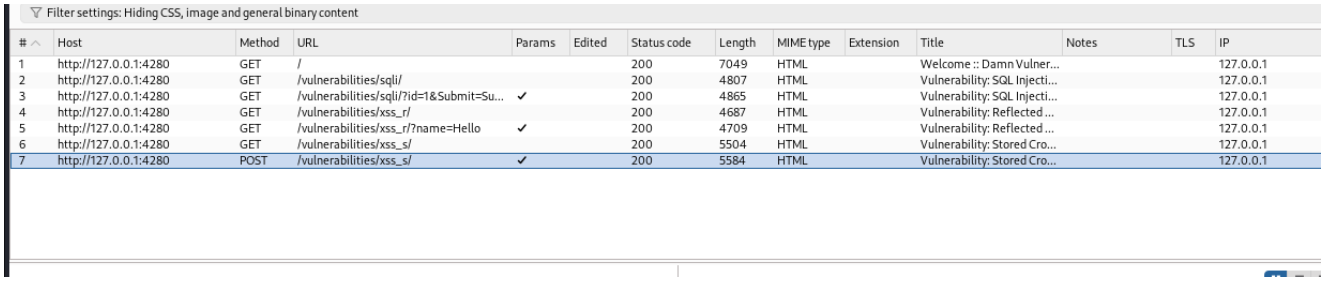
Phase	Procedure	Evidence
Preparation	Start DVWA and verify browser access	Local DVWA page
Mapping	Browse key routes via Burp Proxy; capture GET/POST requests	HTTP history
Manual testing	Replay and modify parameters using Burp Repeater	Requests and responses
Authentication	Submit invalid credentials and inspect redirect/login error	302 and login failed
Discovery	Use ZAP Manual Explore then Spider	122 URLs / 57 nodes
Reporting	Document observed behavior, impact and remediation	This report

Evidence handling

Screenshots are selected for readability and may be cropped to avoid disclosing session identifiers. Request payloads are limited to the authorized local DVWA lab. When the same endpoint returns different records, the behavior is documented without asserting an authorization defect unless access control is separately tested.

04 | Burp Suite: Interception and Mapping

The Burp Proxy HTTP History captured requests to the DVWA home page, SQL injection page, reflected XSS page, and stored XSS page. A later guestbook submission generated a POST request to /vulnerabilities/xss_s/.



Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP
1	http://127.0.0.1:4280	GET	/			200	7049	HTML		Welcome :: Damn Vulner...			127.0.0.1
2	http://127.0.0.1:4280	GET	/vulnerabilities/sqli/			200	4807	HTML		Vulnerability: SQL Injecti...			127.0.0.1
3	http://127.0.0.1:4280	GET	/vulnerabilities/sqli/?id=1&Submit=Su...	✓		200	4865	HTML		Vulnerability: SQL Injecti...			127.0.0.1
4	http://127.0.0.1:4280	GET	/vulnerabilities/xss_r/			200	4687	HTML		Vulnerability: Reflected ...			127.0.0.1
5	http://127.0.0.1:4280	GET	/vulnerabilities/xss_r/?name=Hello	✓		200	4709	HTML		Vulnerability: Reflected ...			127.0.0.1
6	http://127.0.0.1:4280	GET	/vulnerabilities/xss_s/			200	5504	HTML		Vulnerability: Stored Cro...			127.0.0.1
7	http://127.0.0.1:4280	POST	/vulnerabilities/xss_s/	✓		200	5584	HTML		Vulnerability: Stored Cro...			127.0.0.1

Evidence 1: Burp HTTP History with the guestbook POST request selected.

Key input fields

Endpoint	Method	Inputs
/vulnerabilities/sqli/	GET	id, Submit
/vulnerabilities/xss_r/	GET	name
/vulnerabilities/xss_s/	POST	txtName, mtxMessage, btnSign
/login.php	POST	username, password, Login, user_token

05 | Finding F-01: SQL Injection

Classification	SQL injection | manually demonstrated
Lab severity	High (training-context assessment)
Affected path	/vulnerabilities/sqli/
Security setting	DVWA Low

Observation

An input supplied to the DVWA SQL injection exercise changed the application response so that multiple sample users were returned rather than a single requested record. This demonstrates unsafe handling of query input in the deliberately vulnerable lab.

Impact

In a real application, a similar flaw can expose database records, compromise confidentiality and potentially enable further database manipulation depending on permissions. No production system was tested.

Recommendation

• Use parameterized SQL queries/prepared statements.

• Validate input types and enforce least-privilege database permissions.

• Retest with the same input after applying the fix.

Evidence: earlier manual SQLi and Burp response observations; detailed raw exploit payloads are omitted from this public-facing summary.

06 | Finding F-02: Reflected XSS

Classification	Reflected cross-site scripting
Lab severity	Medium to High, context-dependent
Affected path	/vulnerabilities/xss_r/
Security setting	DVWA Low

Observation

A JavaScript test string supplied through the name parameter was reflected in the server response and executed in the browser, producing an alert. The result indicates insufficient context-aware output encoding in the vulnerable training page.

Impact

In real applications, reflected XSS may allow malicious script to run in the origin context if a victim follows a crafted link. The practical demonstrated alert execution only; it did not demonstrate account takeover.

Recommendation

• Apply context-aware HTML output encoding and avoid rendering untrusted input as executable markup.

• Use safe DOM APIs and apply Content Security Policy as defense in depth.

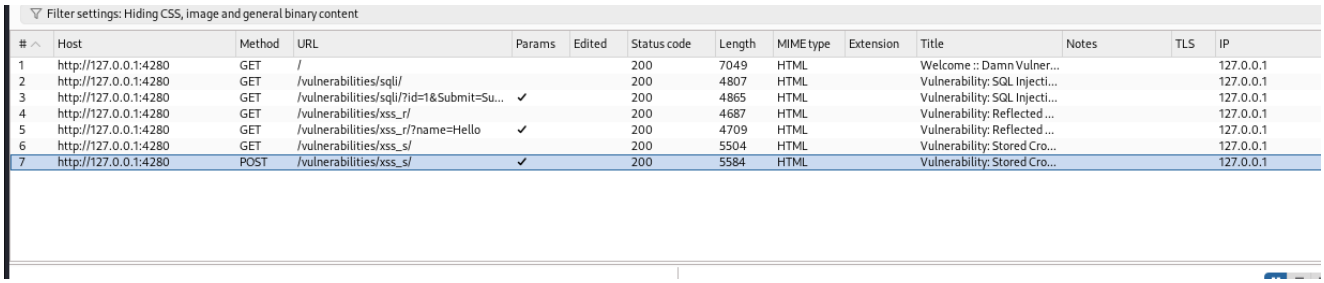
• Retest the reflected input after remediation.

07 | Finding F-03: Stored XSS

Classification	Stored cross-site scripting
Lab severity	High (training-context assessment)
Affected path	/vulnerabilities/xss_s/
Security setting	DVWA Low

Observation

A script-bearing guestbook message was accepted and stored by the training application. When the guestbook content was rendered, the script executed and triggered a browser alert. A later benign POST submission confirmed the guestbook parameters used for the same feature.



Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP
1	http://127.0.0.1:4280	GET	/			200	7049	HTML		Welcome :: Damn Vulner...			127.0.0.1
2	http://127.0.0.1:4280	GET	/vulnerabilities/sqli/			200	4807	HTML		Vulnerability: SQL Injecti...			127.0.0.1
3	http://127.0.0.1:4280	GET	/vulnerabilities/sqli/?id=1&Submit=Su...	✓		200	4865	HTML		Vulnerability: SQL Injecti...			127.0.0.1
4	http://127.0.0.1:4280	GET	/vulnerabilities/xss_r/			200	4687	HTML		Vulnerability: Reflected ...			127.0.0.1
5	http://127.0.0.1:4280	GET	/vulnerabilities/xss_r/?name=Hello	✓		200	4709	HTML		Vulnerability: Reflected ...			127.0.0.1
6	http://127.0.0.1:4280	GET	/vulnerabilities/xss_s/			200	5504	HTML		Vulnerability: Stored Cro...			127.0.0.1
7	http://127.0.0.1:4280	POST	/vulnerabilities/xss_s/	✓		200	5584	HTML		Vulnerability: Stored Cro...			127.0.0.1

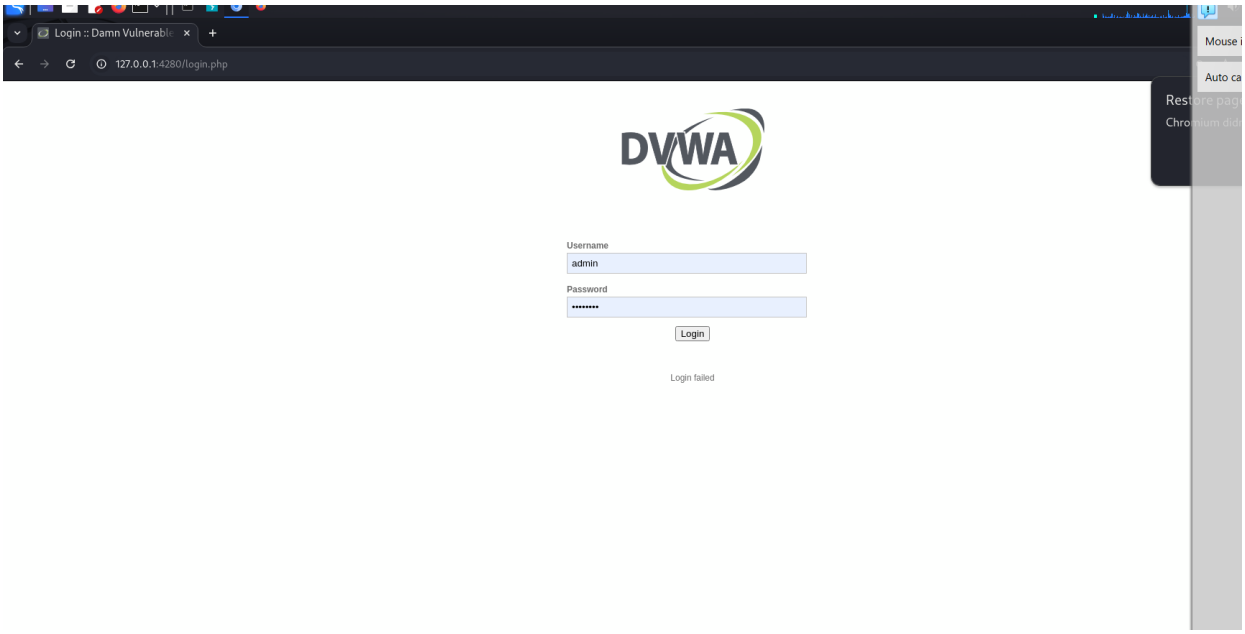
Evidence 2: POST /vulnerabilities/xss_s/ and input parameter mapping (benign example).

Recommendation

Encode user-generated content when rendering it, sanitize only where HTML input is explicitly supported, and use a restrictive CSP as an additional safeguard. Verify persistence and safe rendering after remediation.

08 | Authentication Failure Test

The invalid-login test submitted an incorrect password for the training account. Burp recorded POST /login.php with a 302 redirect back to /login.php, followed by a GET request that returned 200 OK. The browser visibly displayed “Login failed.”



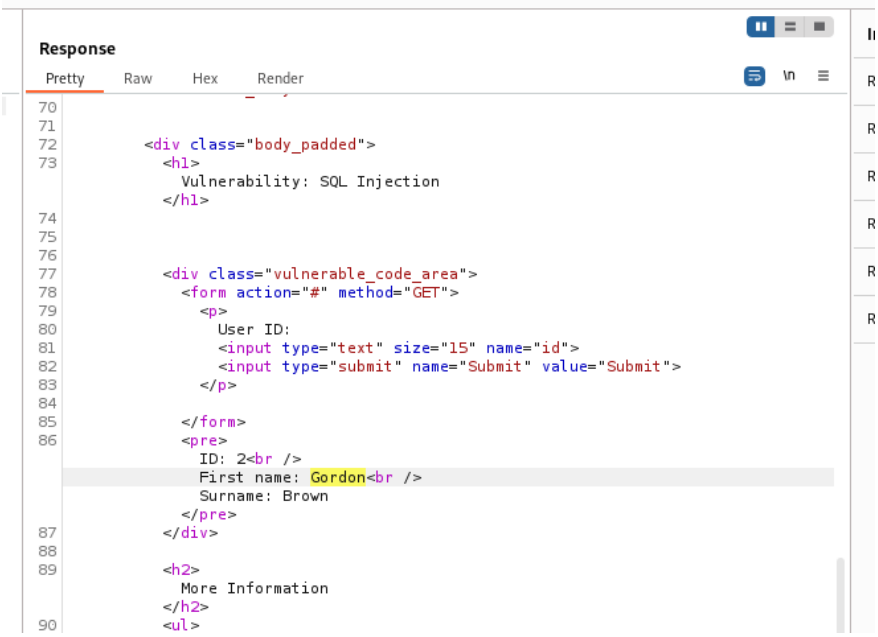
Evidence 3: Login failed message shown in the DVWA browser.

Conclusion

The tested incorrect password was rejected. This is expected authentication behavior, not a confirmed authentication bypass or vulnerability. No password guessing campaign or account lockout assessment was conducted.

09 | Parameter Tampering: Repeater

The SQL injection training page was used to observe how a changed request parameter affects the response. Burp Repeater first sent id=1, then id=2. The response to id=2 displayed First name: Gordon and Surname: Brown.



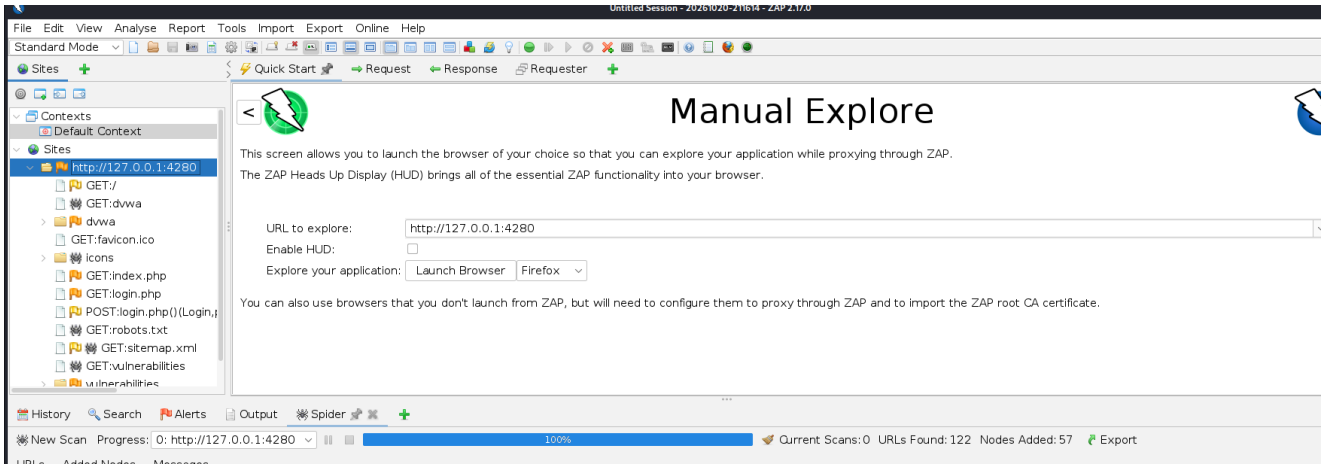
Evidence 4: Response content for id=2 shows the Gordon Brown sample record.

Interpretation

This confirms successful request modification and server-side parameter processing. It does not independently establish an insecure direct object reference (IDOR) or broken access control, because the DVWA exercise may intentionally allow access to those sample records.

10 | ZAP Discovery Results

OWASP ZAP Manual Explore collected site paths. A subsequent Spider scan against the local DVWA endpoint completed without a running scan remaining.



Evidence 5: ZAP Spider completed at 100%, with 122 URLs found and 57 nodes added.

Metric	Observed value
Target	http://127.0.0.1:4280
Progress	100%
URLs found	122
Nodes added	57
Current scans at capture	0

11 | Passive Observations & Limitations

ZAP also showed passive alert indicators during exploration. These are leads for review, not verified vulnerabilities. The previously observed alert categories included missing security headers, cookie attributes, server information disclosure, and possible directory browsing.

Validation checklist

• Inspect the exact affected URL and raw response before reporting a security-header issue.

• Verify cookie flags against the relevant authentication cookie and browser context.

• Distinguish a directory listing response from an error page or intended static index.

• Record confidence and reproducibility; do not count every scanner observation as a distinct exploit.

Limits

An earlier ZAP Active Scan was interrupted when the environment became unresponsive. The completed 122-URL Spider scan is a discovery scan, not proof of vulnerability. Testing was performed on DVWA Low, so findings should not be generalized to production configurations.

12 | Risk Reduction & Retesting Plan

Priority	Action	Validation
1	Parameterized SQL statements and restricted DB privileges	Repeat SQLi input; confirm no extra records
1	Encode stored guestbook content at output	Reload guestbook; ensure script does not execute
2	Encode reflected input in its HTML context	Repeat reflected input; verify safe display
2	Review auth/session cookies and security headers	Inspect raw HTTP responses
3	Harden error handling and directory indexes	Verify directory URLs and error output

Recommended verification process

Capture the baseline request, apply one fix at a time, repeat the same input under the same DVWA configuration, compare response bodies, and record screenshots of the remediated behavior. Avoid publishing active session tokens or passwords in evidence.

13 | Conclusion, Deliverables & References

The Week 7 DVWA lab demonstrated three common web application vulnerabilities through manual testing: SQL injection, reflected XSS and stored XSS. Burp Suite captured and replayed requests, while OWASP ZAP provided application mapping and Spider-based endpoint discovery. Authentication failure and parameter modification were also documented without overclaiming security impact.

Assignment completion

Task	Status
22 | Burp Suite setup and interception	Complete
23 | Recon and application mapping	Complete
24 | Manual vulnerability testing	Complete
25 | Directory and endpoint discovery	Complete
26 | Findings and final report	Complete

References for further study

• OWASP Web Security Testing Guide (WSTG) — testing methodology.

• OWASP Cross Site Scripting Prevention Cheat Sheet.

• OWASP SQL Injection Prevention Cheat Sheet.

• PortSwigger Web Security Academy — Burp Suite learning resources.

Prepared for DG Interns Hub Cybersecurity Internship | Submitted by Akash Sanuria | DG/SEPTEMBER/CYBER/171